

1 - Błędy w implementacji

Przy okazji doczytywania w badaniu, zauważyłem że wkradł się błąd implementacyjny. Mocno skupiliśmy się na używaniu macierzy jednostkowej w liczeniu wektora spadku, a okazuje się że nie jest to jedyne miejsce w którym występuje theta. W minimalizacji podprzestrzeni nadal używaliśmy (niewprost) macierzy jednostkowej. Dodałem tam korzystanie z naszej macierzy hesjanu, to dało zauważalne przyspieszenie.

Teraz przy podaniu początkowej macierzy wykorzystujemy ją w każdym fragmencie algorytmu.

2 - Dlaczego tylko diagonalą?

Zastanawialiśmy się dlaczego korzystamy przy zastąpieniu macierzy jednostkowej korzystaliśmy jedynie z diagonali, a nie z całej macierzy. Okazuje się że to jednak ma swoje konsekwencje.

2.1

Gdy liczymy punkt Cauchy'ego, normalnie liczymy pochodną B po d: $\theta * d'B_0 d - d' W M W' d$, dla pełnej macierzy wykonujemy iloczyn macierz wektor co daje nam n^2 .

2.2

Gdy minimalizujemy podprzestrzeń, we wzorze Woodbury'ego, też dzielimy wektory przez tą naszą diagonalę (lub macierz) przeskalowaną przez theta

2.3

Przy aktualizacji par korekcyjnych, też iloczyny macierz-wektor w przypadku pełnej macierzy

Zastosowanie całej macierzy wydają się na wielu etapach mocno zwiększać koszt obliczeń.

3 - Rotacje funkcji testowych

W celu sprawdzenia kiedy zastosowanie całej macierzy może być korzystne (lub kiedy bazowanie na samej diagonalu może być szkodliwe) przygotowałem takie funkcję testowe:

Wyliczyłem takie macierze rotacji:

```
=== uniform_45 ===
[[ 0.7071 -0.7071  0.      0.      0.      0.      0.      0.      0.
 0.      ]
 [ 0.5     0.5    -0.7071  0.      0.      0.      0.      0.      0.
 0.      ]
 [ 0.3536  0.3536  0.5     -0.7071  0.      0.      0.      0.      0.
 0.      ]
 [ 0.25     0.25   0.3536  0.5     -0.7071  0.      0.      0.      0.
 0.      ]
 [ 0.1768  0.1768  0.25    0.3536  0.5     -0.7071  0.      0.      0.
 0.      ]
 [ 0.125    0.125  0.1768  0.25    0.3536  0.5     -0.7071  0.      0.]
```

```

0.    ]
[ 0.0884  0.0884  0.125  0.1768  0.25  0.3536  0.5  -0.7071  0.
0.    ]
[ 0.0625  0.0625  0.0884  0.125  0.1768  0.25  0.3536  0.5  -0.7071
0.    ]
[ 0.0442  0.0442  0.0625  0.0884  0.125  0.1768  0.25  0.3536  0.5
-0.7071]
[ 0.0442  0.0442  0.0625  0.0884  0.125  0.1768  0.25  0.3536  0.5
0.7071]]
Orthogonal check (R R' = I): max error = 2.78e-17
det(R) = 1.0000

=== golden ===
[[-0.7373 -0.6756  0.      0.      0.      0.      0.      0.      0.
0.    ]
[ 0.0589 -0.0643  0.9962  0.      0.      0.      0.      0.      0.
0.    ]
[-0.4097  0.4471  0.0531 -0.7934  0.      0.      0.      0.      0.
0.    ]
[ 0.5258 -0.5738 -0.0681 -0.5995  0.1736  0.      0.      0.      0.
0.    ]
[ 0.0782 -0.0853 -0.0101 -0.0892 -0.8306  0.5373  0.      0.      0.
0.    ]
[ 0.0129 -0.0141 -0.0017 -0.0147 -0.137  -0.2183 -0.9659  0.      0.
0.    ]
[ 0.0222 -0.0242 -0.0029 -0.0253 -0.236  -0.3762  0.1195  0.887  0.
0.    ]
[ 0.0401 -0.0438 -0.0052 -0.0457 -0.426  -0.679  0.2157 -0.4339 -0.342
0.    ]
[-0.0135  0.0147  0.0017  0.0154  0.1433  0.2283 -0.0725  0.1459 -0.8682
-0.3827]
[ 0.0056 -0.0061 -0.0007 -0.0064 -0.0593 -0.0946  0.03  -0.0604  0.3596
-0.9239]]
Orthogonal check (R R' = I): max error = 2.22e-16
det(R) = 1.0000

=== random ===
[[ 0.0909 -0.5112  0.2211  0.5329 -0.3343  0.0353  0.1921  0.203  -0.4256
0.1593]
[ 0.2624  0.2968  0.1433  0.6909  0.2092 -0.1257 -0.1786  0.2145  0.4337
-0.1398]
[-0.0552 -0.305  0.4387 -0.1875 -0.043  -0.2788  0.3856 -0.0538  0.6089
0.2727]
[ 0.639  -0.3559 -0.1744 -0.1737  0.528  -0.102  -0.1407  0.0668 -0.0913
0.2874]
[ 0.2218  0.1973 -0.1956  0.2461  0.0319  0.4462  0.4721 -0.5601  0.0745
0.2661]
[ 0.0863  0.2737 -0.5389 -0.0458 -0.3206 -0.2645  0.2951  0.4916  0.0783
0.3427]
[-0.5021 -0.0271 -0.0322  0.1513  0.2074  0.2445 -0.3795  0.0897  0.0812
0.6796]

```

```

    [-0.3806 -0.4329 -0.5565  0.267   0.2595 -0.2641  0.1529 -0.1712  0.1415
    -0.2774]
    [ 0.1363 -0.3456 -0.2042 -0.1109 -0.2587  0.6343 -0.1465  0.2769  0.4393
    -0.2181]
    [ 0.1985 -0.0976 -0.163   0.0652 -0.5317 -0.3063 -0.5137 -0.4858  0.1425
    0.1532]]
    Orthogonal check (R R' = I): max error = 8.88e-16
    det(R) = 1.0000

```

Odpowiadają one odpowiednio trzem strategiom rotacji:

Uniform 45° - łańcuch rotacji w kolejnych płaszczyznach (0,1), (1,2)... każda o kąt 45°

Golden angle - łańcuch rotacji z różnymi kątami $(k+1) * 137.5^\circ$ dla płaszczyzny k. Miało to na celu wykluczyć powtarzalność w rotacjach, z uwagi na brak zwielokrotniania się kątów w wielokrotnościach.

Random - Wygenerowałem macierz $n \times n$ z rozkładem normalnym, poddałem rozkładowi QR i zastosowałem czynnik Q jako naszą macierz obrotu. Wydają mi się że dzięki temu powinniśmy zostać prawdziwie losową zrotowaną funkcję przy zachowaniu ortogonalności macierzy i wyznacznika 1.

Dla hesjanów tych obróconych funkcji możemy policzyć frakcję energii diagonalnej na poziomie:

typ rotacji	Frakcja diag. (10D)	Frakcja diag. (50D)
Brak	1.00	1.00
Uniform 45°	0.49	0.91
Golden angle	0.83	0.94
Random	0.26	0.18

4 - Eksperymenty z pełnym hesjanem

4.1 - Zmiany w implementacji

Aby umożliwić eksperymenty z hesjanem dodałem abstrakcję nad początkowym paremetrem pełniącym rolę przekątnej hesjanu, hesjanu, lub macierzy jednostkowej. Teraz tworzymy obiekt początkowy w odpowiednim sposób, ale algorytm w każdym wariancie działa tak samo.

4.2 - Wyniki

Uruchomiłem nowe poobracane funkcje w dwóch rozmiarach, dla trzech początkowych rodzajów hesjanu:

- **n = 10, m = 10:** gdzie poprawki mogą opisać nam $2m = 20$ kierunków, a więc pokrywa nasze 10D co sprawia że początkowy hesjan jest mało ważny, ponieważ poprawki w pełni uczą się kształtu funkcji
- **n = 50, m = 5:** Tutaj zakładałem, że przy znacząco mniejszym rozmiarze okna historii od wymiaru funkcji, dobra aproksymacja hesjanu może mieć zdecydowanie większe znaczenie.

n = 50, m = 5



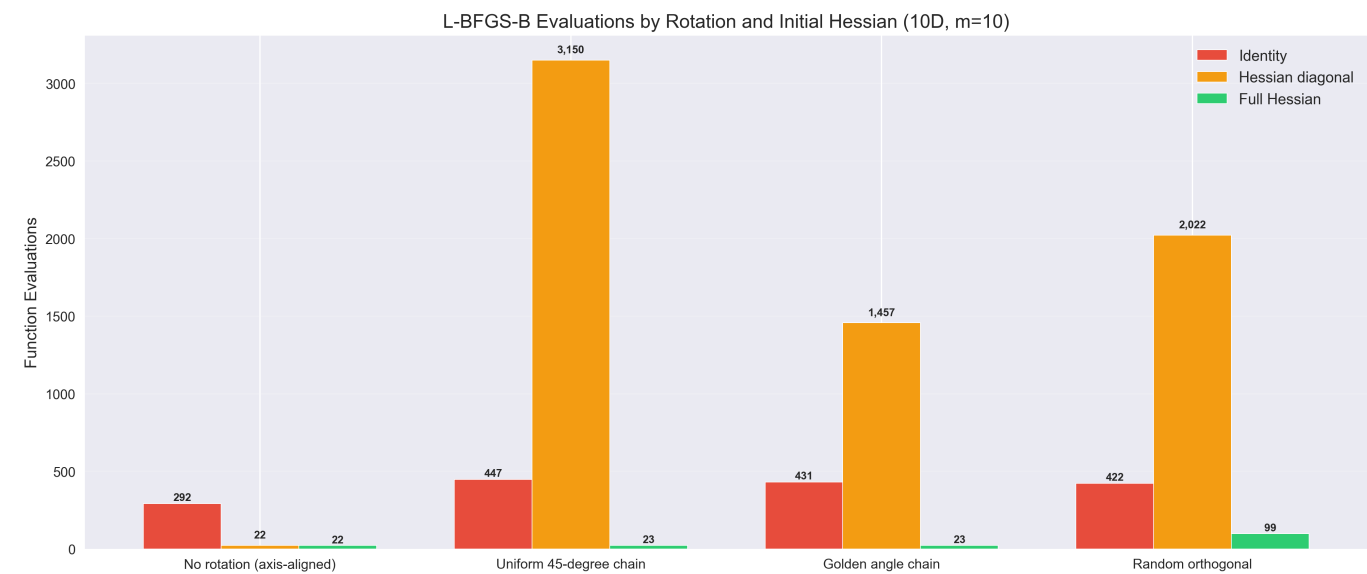
Obrót	M. Jednostkowa	Przekątna	Pełny Hesjan
None (axis-aligned)	5,064	23	23
Uniform 45°	3,508	10,000+	22
Golden angle	5,968	10,000+	22
Random	6,111	10,000+	23

Widzimy że pełny Hesjan sprawia że wchodzimy w margines minimum w bardzo szybkim czasie, niezależnie od rotacji.

Diagonała oczywiście, zawierając de facto błędne informację, robi nam zdecydowanie pod górkę.

Macierz jednostkowa reprezentuje klasyczne zachowanie uczenia się kształtu funkcji od zera.

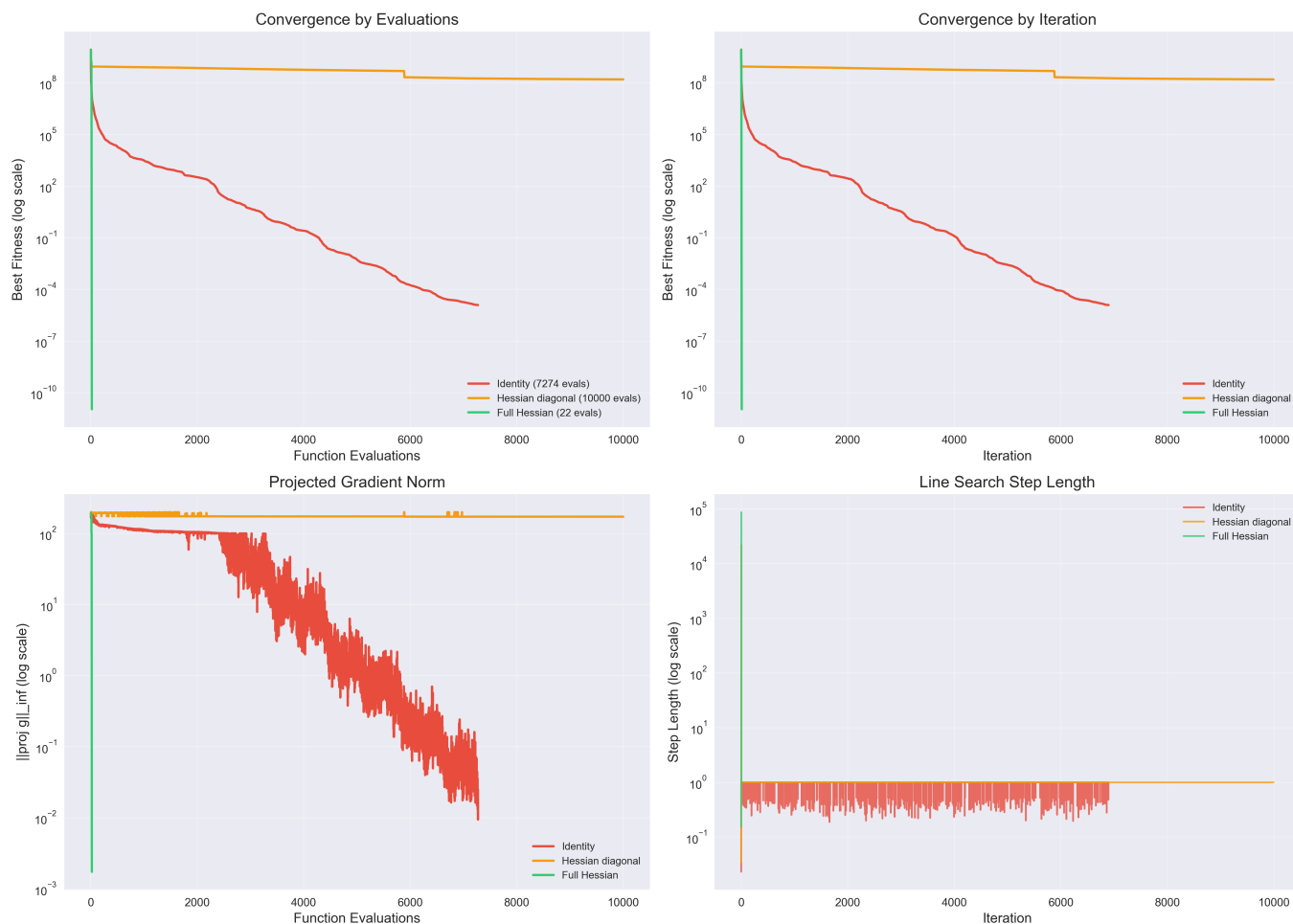
n = 10, m = 10



Gdy poprawki obejmują całą przestrzeń pełny hesjan nadal daje przewagę, ale nie aż tak rażącą.

Zbieżności 50D, obrót *Random*

L-BFGS-B Initial Hessian Comparison
Random orthogonal (50D, m=5)



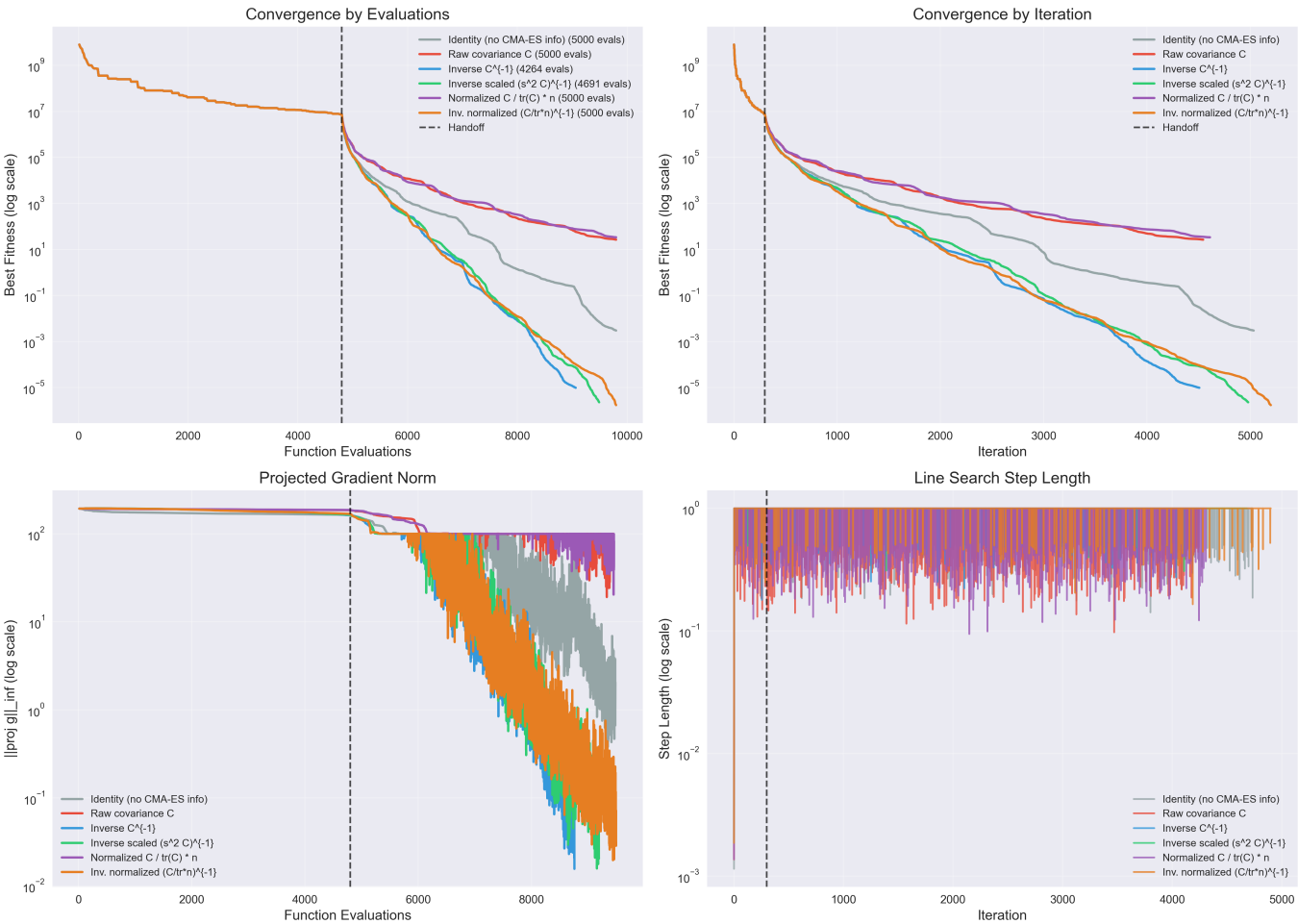
5 - Eksperymenty CMA-ES -> L-BFGS

Głowiliśmy się, dlaczego jednak podajemy odwrotność macierzy kowariancji z CMAES-a. To też potwierdził się w eksperymentach poniżej. Otóż problem tkwił w różnicy między L-BFGS i L-BFGS-B: W wersji bez ograniczeń faktycznie w praktyce trzymamy odwrotność hesjanu, jednak w wersji ograniczonej, przetrzymujemy aproksymację faktycznego hesjanu i używamy jej do liczenia punkty Cauchyego, a odwracamy go "na życzenie".

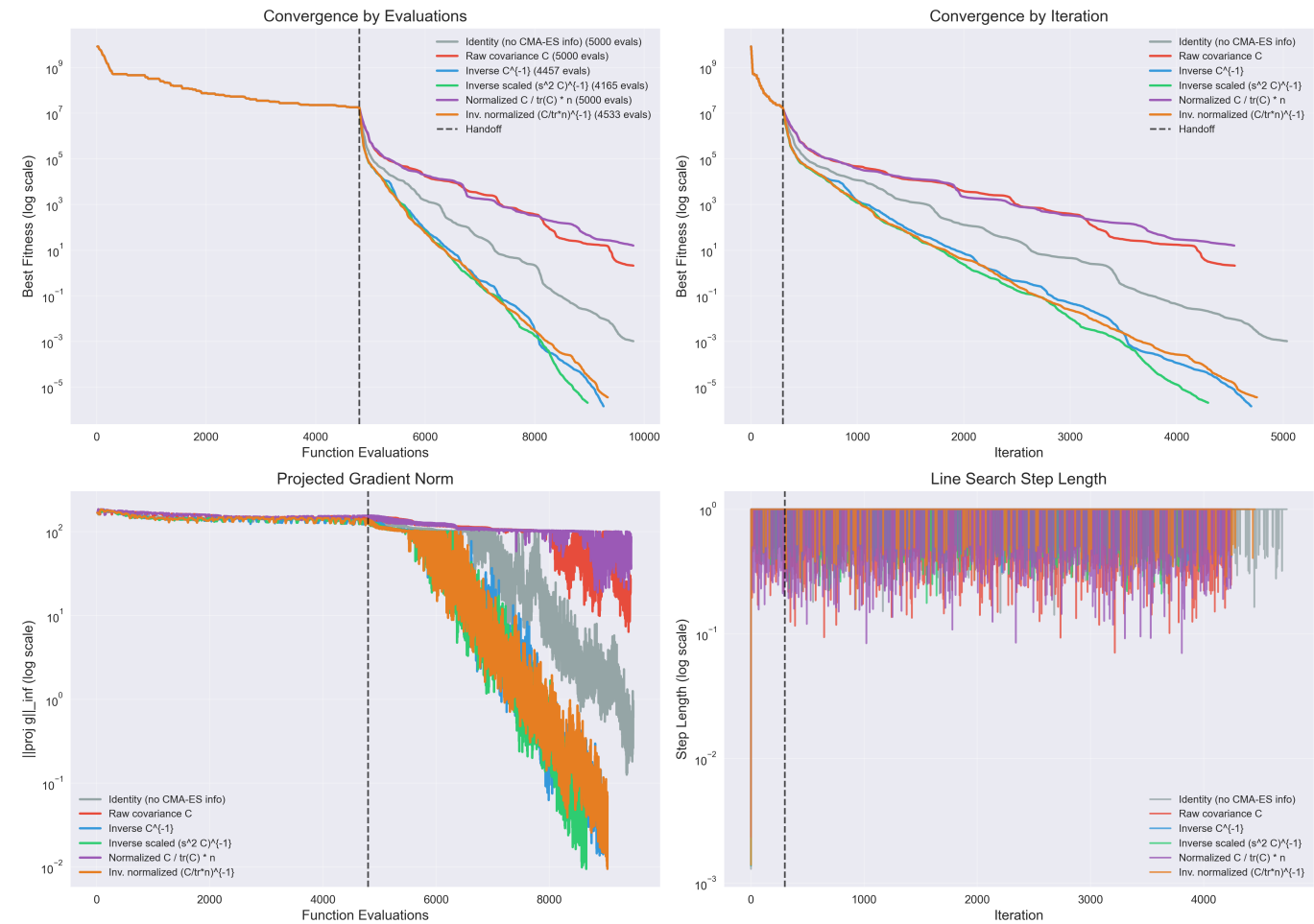
Zgodnie z planem testowałem 6 transformacji: macierz jednostkowa, bezpośrednio C, odwrotność C^{-1} , odwrotność przeskalowana $(\sigma^2 C)^{-1}$, znormalizowana $C/\text{tr}(C)*n$ oraz odwrócona znormalizowana $(C/\text{tr}(C)*n)^{-1}$.

50D, m=5, 300 generacji CMA-ES

CMA-ES -> L-BFGS-B: Covariance Transformations
Uniform 45-degree chain (50D, m=5, 300 CMA-ES gen)

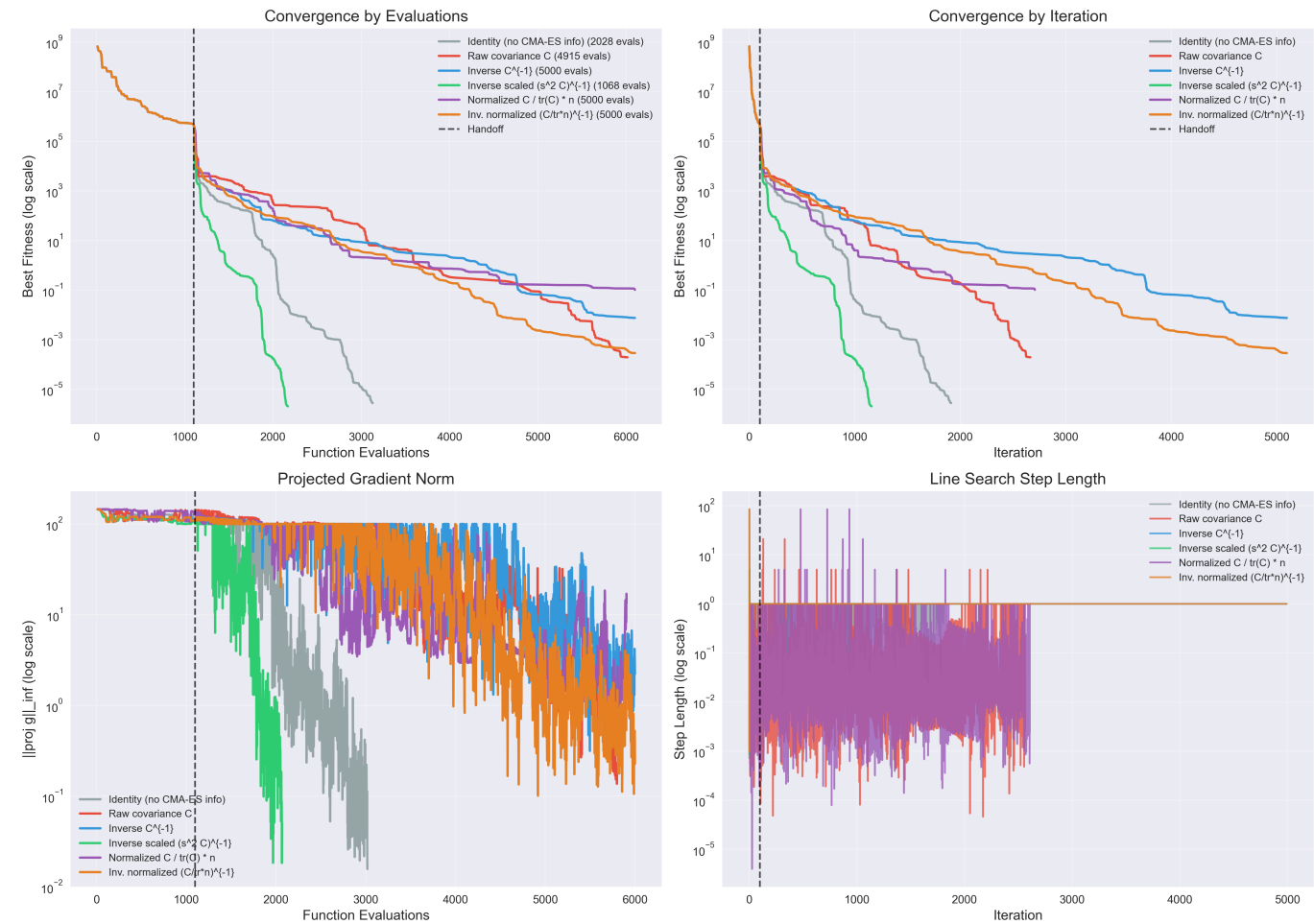


CMA-ES -> L-BFGS-B: Covariance Transformations
Random orthogonal (50D, m=5, 300 CMA-ES gen)

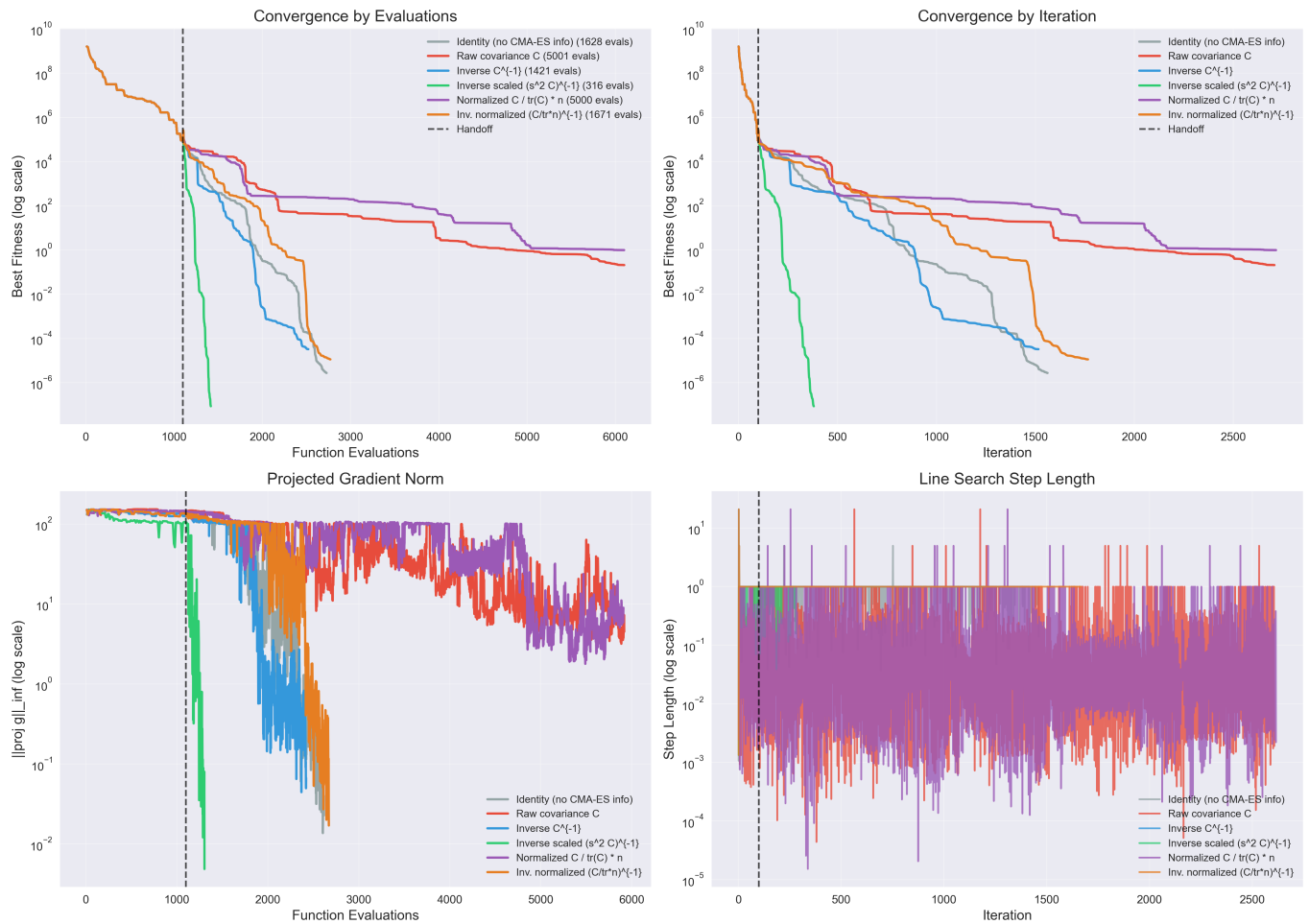


10D, m=5, 100 generacji CMA-ES

CMA-ES -> L-BFGS-B: Covariance Transformations
Uniform 45-degree chain (10D, m=5, 100 CMA-ES gen)



CMA-ES -> L-BFGS-B: Covariance Transformations
Random orthogonal (10D, m=5, 100 CMA-ES gen)



Jak widać faktycznie potrzebujemy odwróconej macierzy żeby osiągnąć poprawę.

Skalowanie prawie nie ma znaczenie, theta i tak dostosowuje skalę globalną w kilku iteracjach